Understanding JSON vs Protocol Buffers (Protobuf)

Lecture Notes for First-Year Students

Learning Objectives

• Understand why applications need serialization.

• Compare JSON and Protocol Buffers.

• Learn how Protobuf converts data into compact binary.

• Understand field numbers, wire types and Varints at a conceptual level.

1. Why Do Computers Need a Common Language?

When two applications communicate over a network, they cannot exchange Java or Python objects directly. The sender must convert the object into a transferable format (serialization). The receiver reconstructs the original object (deserialization). JSON and Protocol Buffers are two popular serialization formats.

2. JSON Example

```
{
  "orderId":101,
  "customerName":"Fayaz",
  "product":"Laptop",
  "quantity":2,
  "price":75000,
  "isPaid":true
}
```

JSON is human-readable because it transmits both field names and values.

Problem: If one million orders are sent, the field names (orderId, customerName, etc.) are also transmitted one million times.

3. The Protobuf Solution

Field No.	Field Name	Type
1	orderId	int32
2	customerName	string
3	product	string
4	quantity	int32
5	price	int32
6	isPaid	bool

Instead of repeatedly sending field names, both applications agree on this schema in advance. Only the field number and value are transmitted.

```
Conceptual Representation
1 -> 101
```

```
2 -> Fayaz
3 -> Laptop
4 -> 2
5 -> 75000
6 -> true
```

Important: This is only for understanding. The network never sends this text.

4. What Actually Travels Over the Network?

```
08 65
12 05 46 61 79 61 7A
1A 06 4C 61 70 74 6F 70
20 02
28 F8 C9 04
30 01
```

These are hexadecimal bytes representing the binary Protobuf message.

5. Breaking Down One Field

orderId = 101

Field Number = 1
Wire Type (int32) = 0
Formula: (Field Number << 3) | Wire Type
(1 << 3) | 0 = 8 decimal = 0x08
Value 101 = 0x65

Final Bytes: 08 65

customerName = "Fayaz"

Field Number = 2, Wire Type = 2 (length-delimited).
(2 << 3)|2 = 18 decimal = 0x12.
Next byte = string length = 5 (0x05).
Characters: F=46, a=61, y=79, a=61, z=7A.
Final Bytes: 12 05 46 61 79 61 7A

Why is 75000 encoded as F8 C9 04?

Integers are stored using Varint encoding. Small numbers use fewer bytes, reducing message size. Large numbers are split into multiple 7-bit groups, producing compact binary such as F8 C9 04.

6. Real-Life Analogy

Imagine a classroom. JSON is like writing every student's name, department, and roll number on every attendance sheet. Protobuf assigns each student a roll number once. Afterwards, the attendance sheet contains only the roll number because everyone already knows the mapping.

7. JSON vs Protobuf Comparison

Feature	JSON	Protobuf
Readable	Yes	No (Binary)
Message Size	Large	Small
Speed	Good	Excellent

Schema Required	No	Yes
Common Usage	REST APIs	gRPC, Microservices

Key Takeaways

• JSON is text-based and human-readable.

• Protobuf is binary and machine-optimized.

• Field names are replaced by field numbers defined in a schema.

• Smaller messages mean faster communication and lower bandwidth.

• gRPC uses Protocol Buffers by default.